Notebook 05: **Multi-Frame Semantic Consistency Over Motion**

1. Purpose of This Notebook

Notebook 01 established that the SemanticKITTI loading pipeline is correct and numerically trustworthy. Notebook 02 showed that a simple range-aware pseudo-reflectivity proxy can behave differently from raw intensity and can become more semantically informative on a single frame. Notebook 03 extended that story through motion and showed that the proxy remains temporally stable and visually structured across a short frame window. Notebook 04 then attached the signal directly to semantic labels and showed that class-wise behavior can become modestly more interpretable under the proxy.

The purpose of Notebook 05 is to connect those threads into one disciplined temporal-semantic test:

do the class-wise patterns observed in Notebook 04 persist across a short motion window, or were they just a one-frame lucky strike?

This notebook is still part of the preliminary stage, so it must stay small, sharp, and honest. It is not meant to become a full final-project substitute. Its job is to validate short-window semantic consistency, not to exhaust the entire project arc.

2. Conceptual Motivation

A useful robotic perception cue should not only look meaningful on one frozen frame. It should remain at least somewhat stable as the platform moves, viewpoints change, near and far surfaces shift, and the scene sweeps through the field of view. Notebook 03 already made this argument for the signal globally. Notebook 05 now asks the same question at the **semantic class level**.

In Notebook 04, we observed that different semantic classes can occupy different signal regimes, and that pseudo-reflectivity can reshape class-wise behavior in a more interpretable way than raw intensity alone. That observation is interesting, but one frame can still lie with a straight face. So the next logical step is to test whether those class-level relationships remain visible over a short sequence.

The key research attitude here is restrained: we are not claiming true physical reflectivity recovery, we are not training a semantic segmentation model, and we are not yet making broad sequence-level claims. We are only asking whether a lightweight range-aware augmentation signal shows **short-horizon semantic consistency through motion**.

3. Mathematical Model Used in This Notebook

Each LiDAR point is represented as:

$$p_i = (x_i, y_i, z_i, I_i)$$

with semantic label

$$s_i \in C$$

where (x_i, y_i, z_i) are spatial coordinates, I_i is raw LiDAR intensity, and s_i is the semantic class label.

For each point, we compute Euclidean range:

$$R_i = \sqrt{x_i^2 + y_i^2 + z_i^2}$$

and use the same practical pseudo-reflectivity proxy as in the earlier notebooks:

$$\hat{\rho}_i = I_i \cdot R_i$$

This is **not** calibrated physical reflectivity. It is a heuristic range-aware proxy whose value is interpretability, not exact reflectance recovery. That honesty clause remains non-negotiable.

A simplified LiDAR return interpretation is:

$$I_i \propto \frac{\rho_i \cos(\alpha_i)}{R_i^2}$$

which reminds us why raw intensity is entangled with geometry and incidence effects. The broader reflectivity literature makes exactly this point: calibrated reflectivity is more interpretable than raw intensity because raw intensity is affected by range and angle of incidence. Since full calibration is outside current scope, this notebook keeps using the lightweight proxy instead.

4. What This Notebook Will Do

This notebook will follow the same disciplined structure as the earlier notebooks, but now with semantics carried through motion.

Step A: Select a short consecutive frame window
We will use a small contiguous sequence, consistent with the preliminary-stage scope, rather than scaling to a large final-stage study.

Step B: Load each frame and its semantic labels
For every selected frame, we will recover point coordinates, raw intensity, range, pseudo-reflectivity, and semantic labels.

Step C: Restrict analysis to sufficiently supported classes
We will only keep classes with enough points per frame so that the temporal class-wise statistics are not built on tiny, noisy scraps.

Step D: Build frame-wise class summaries
For each frame and each valid class, we will compute compact summaries for raw intensity and

pseudo-reflectivity, such as mean, spread, and support size. This turns the semantic scene into a small temporal table instead of a chaos soup.

Step E: Measure semantic consistency across motion
We will inspect whether class ordering, class-wise means, and class-wise separability patterns remain reasonably stable over the selected motion window. This is the core test of the notebook. It directly matches the roadmap item that says the preliminary Notebook 05 should validate whether the class-wise patterns from Notebook 04 persist across a short sequence.

Step F: Visualize semantic behavior over time
We will generate compact temporal plots showing how class-wise raw intensity and class-wise pseudo-reflectivity evolve across frame index, and whether the relative advantage of the proxy stays stable, weakens, flips, or behaves in a scene-dependent way. This is still preliminary, so the goal is clarity, not maximal plot spam.

5. What We Are Trying to Achieve

This notebook is trying to achieve four specific things.

First, it tests whether the semantic story from Notebook 04 survives motion at all. If the class-wise patterns collapse immediately, then the one-frame result was fragile.

Second, it checks whether pseudo-reflectivity behaves like a reusable class-aware cue rather than a one-frame gimmick.

Third, it provides the correct endpoint for the preliminary stage: enough evidence to justify deeper evaluation, without consuming the heavier final-stage work that should still cover broader frame ranges, stronger metrics, proxy comparison, and failure-case analysis.

Fourth, it tightens the research arc: Notebook 02 asked whether a range-aware signal helps on one frame, Notebook 03 asked whether the signal survives motion globally, Notebook 04 asked whether the signal has semantic class structure on one frame, and Notebook 05 asks whether that semantic class structure persists through motion.

6. Expected Outputs

By the end of this notebook, we expect to have:

- a short labeled multi-frame sequence loaded correctly,
- a frame-wise class summary table for raw intensity and pseudo-reflectivity,
- temporal plots of class-wise signal behavior across motion,
- a compact consistency analysis of whether major class patterns persist,
- and a short conclusion about whether the semantic effect is stable, weakly stable, or clearly scene-dependent.

7. Expected Outcome

If everything behaves reasonably well, the final takeaway of Notebook 05 should be something like this:

The range-aware pseudo-reflectivity proxy does not act like true calibrated reflectivity, but it preserves at least part of its class-wise semantic structure across a short motion window. Its usefulness is therefore not uniformly dominant, yet it appears stable enough to justify broader final-stage temporal-semantic evaluation.

That is exactly the right scale of claim for preliminary work: not fake-grand, not sleepy, not overcooked. Just enough evidence to say the idea survives contact with motion and deserves a harder final-stage beating.

Step 1: Setup for multi-frame semantic consistency

In this step, we import the required libraries, define the SemanticKITTI sequence paths, and choose a small consecutive frame window for Notebook 05.

The goal is to keep the analysis disciplined and preliminary-stage sized.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Optional: cleaner plots / display
plt.rcParams["figure.figsize"] = (8, 4)
plt.rcParams["figure.dpi"] = 120
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 4)

# SemanticKITTI sequence 00 paths
SEQ_DIR = Path("../data/semantickitti_subset/dataset/sequences/00")
VELODYNE_DIR = SEQ_DIR / "velodyne"
LABEL_DIR = SEQ_DIR / "labels"

# Small, disciplined preliminary window
START_FRAME = 0
NUM_FRAMES = 10

frame_ids = [f"{i:06d}" for i in range(START_FRAME, START_FRAME +
NUM_FRAMES)]

print("Sequence directory :", SEQ_DIR.resolve())
print("Velodyne dir exists:", VELODYNE_DIR.exists())
print("Label dir exists   :", LABEL_DIR.exists())
print("Selected frame IDs :", frame_ids)
```

```
Sequence directory :
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti_subset/
dataset/sequences/00
Velodyne dir exists: True
Label dir exists   : True
```

```
Selected frame IDs : ['000000', '000001', '000002', '000003',
'000004', '000005', '000006', '000007', '000008', '000009']
```

Step 2: Define data-loading utilities

In this step, we define small helper functions to load LiDAR points and semantic labels from SemanticKITTI, and to compute point-wise range and pseudo-reflectivity.

```
def load_frame_points(frame_id, velodyne_dir=VELODYNE_DIR):
    bin_path = velodyne_dir / f"{frame_id}.bin"
    points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
    xyz = points[:, :3]
    intensity = points[:, 3]
    return xyz, intensity

def load_frame_labels(frame_id, label_dir=LABEL_DIR):
    label_path = label_dir / f"{frame_id}.label"
    raw_labels = np.fromfile(label_path, dtype=np.uint32)
    semantic_labels = raw_labels & 0xFFFF
    return semantic_labels

def compute_range_and_pseudo_reflectivity(xyz, intensity):
    ranges = np.linalg.norm(xyz, axis=1)
    pseudo_reflectivity = intensity * ranges
    return ranges, pseudo_reflectivity

# Quick sanity check on the first frame
test_frame = frame_ids[0]
xyz, intensity = load_frame_points(test_frame)
semantic_labels = load_frame_labels(test_frame)
ranges, pseudo_reflectivity =
compute_range_and_pseudo_reflectivity(xyz, intensity)

print("Test frame             :", test_frame)
print("XYZ shape              :", xyz.shape)
print("Intensity shape        :", intensity.shape)
print("Semantic labels shape  :", semantic_labels.shape)
print("Ranges shape           :", ranges.shape)
print("Pseudo-reflectivity    :", pseudo_reflectivity.shape)

print("\nConsistency check")
print("Same number of points across arrays:",
      len(xyz) == len(intensity) == len(semantic_labels) ==
len(ranges) == len(pseudo_reflectivity))
```

```
Test frame             : 000000
XYZ shape              : (124668, 3)
Intensity shape        : (124668,)
Semantic labels shape  : (124668,)
```

```
Ranges shape           : (124668,)
Pseudo-reflectivity    : (124668,)

Consistency check
Same number of points across arrays: True
```

Step 3: Build a compact per-frame summary table

In this step, we load all selected frames and construct a small frame-level summary table containing point count, number of semantic classes, and basic signal statistics. This gives us a quick health check before class-wise temporal analysis.

```
frame_summaries = []

for frame_id in frame_ids:
    xyz, intensity = load_frame_points(frame_id)
    semantic_labels = load_frame_labels(frame_id)
    ranges, pseudo_reflectivity =
compute_range_and_pseudo_reflectivity(xyz, intensity)

    frame_summaries.append({
        "frame_id": frame_id,
        "num_points": len(intensity),
        "num_semantic_classes": len(np.unique(semantic_labels)),
        "intensity_mean": float(np.mean(intensity)),
        "intensity_std": float(np.std(intensity)),
        "range_mean": float(np.mean(ranges)),
        "range_std": float(np.std(ranges)),
        "pseudo_reflectivity_mean":
float(np.mean(pseudo_reflectivity)),
        "pseudo_reflectivity_std": float(np.std(pseudo_reflectivity)),
    })

frame_summary_df = pd.DataFrame(frame_summaries)
frame_summary_df
```

```
  frame_id  num_points  num_semantic_classes  intensity_mean
intensity_std  \
0   000000      124668                    17          0.2941
0.1477
1   000001      124605                    17          0.2946
0.1480
2   000002      124478                    17          0.2942
0.1481
3   000003      124167                    17          0.2957
0.1496
4   000004      123969                    17          0.2956
0.1512
5   000005      123924                    17          0.2947
```

```
0.1532
6   000006      123373                    18          0.2949
0.1562
7   000007      122765                    18          0.2965
0.1599
8   000008      122123                    19          0.2987
0.1611
9   000009      121357                    18          0.3008
0.1616

   range_mean  range_std  pseudo_reflectivity_mean
pseudo_reflectivity_std
0     13.5650    10.5710                    3.6287
2.9401
1     13.5114    10.6273                    3.6067
2.9450
2     13.4012    10.5917                    3.5787
2.9201
3     13.2681    10.5035                    3.5739
2.9210
4     13.1882    10.5035                    3.5544
2.9104
5     13.1344    10.4834                    3.5482
2.9212
6     13.0981    10.5063                    3.5472
2.9204
7     13.0900    10.5474                    3.5697
2.9512
8     13.0589    10.5368                    3.5859
2.9630
9     13.0993    10.6321                    3.6085
3.0023
```

Step 4: Build class-wise statistics for every frame

In this step, we compute class-wise summary statistics for each frame using only classes with enough points to be meaningful.

This creates the temporal semantic table that Notebook 05 is really about.

```
MIN_POINTS_PER_CLASS = 200

class_rows = []

for frame_id in frame_ids:
    xyz, intensity = load_frame_points(frame_id)
    semantic_labels = load_frame_labels(frame_id)
    ranges, pseudo_reflectivity =
compute_range_and_pseudo_reflectivity(xyz, intensity)
```

```
    unique_classes, counts = np.unique(semantic_labels,
return_counts=True)

    for class_id, count in zip(unique_classes, counts):
        if count < MIN_POINTS_PER_CLASS:
            continue

        mask = semantic_labels == class_id

        class_rows.append({
            "frame_id": frame_id,
            "class_id": int(class_id),
            "num_points": int(count),
            "intensity_mean": float(np.mean(intensity[mask])),
            "intensity_std": float(np.std(intensity[mask])),
            "pseudo_reflectivity_mean":
float(np.mean(pseudo_reflectivity[mask])),
            "pseudo_reflectivity_std":
float(np.std(pseudo_reflectivity[mask])),
            "range_mean": float(np.mean(ranges[mask])),
            "range_std": float(np.std(ranges[mask])),
        })

class_summary_df = pd.DataFrame(class_rows).sort_values(
    by=["frame_id", "class_id"]
).reset_index(drop=True)

print("MIN_POINTS_PER_CLASS:", MIN_POINTS_PER_CLASS)
print("Number of retained class-frame rows:", len(class_summary_df))
print("Unique retained classes:",
sorted(class_summary_df["class_id"].unique().tolist()))

class_summary_df.head(20)
```

```
MIN_POINTS_PER_CLASS: 200
Number of retained class-frame rows: 140
Unique retained classes: [0, 10, 40, 44, 48, 50, 51, 52, 60, 70, 71,
72, 80, 99]

   frame_id  class_id  num_points  intensity_mean  intensity_std  \
0    000000         0        2184          0.0470         0.1164
1    000000        10        4234          0.1535         0.1930
2    000000        40       34228          0.2134         0.1134
3    000000        44        3268          0.2999         0.0854
4    000000        48       26360          0.3004         0.0797
5    000000        50       18268          0.4283         0.1488
6    000000        51         370          0.2885         0.1211
7    000000        52        1471          0.3446         0.1598
8    000000        60        1053          0.4085         0.1666
```

9	000000	70	27123	0.3338	0.1312
10	000000	71	1192	0.3057	0.1400
11	000000	72	2964	0.3116	0.1152
12	000000	80	532	0.2034	0.1735
13	000000	99	1227	0.2794	0.1560
14	000001	0	2170	0.0491	0.1302
15	000001	10	4387	0.1530	0.1911
16	000001	40	33233	0.2125	0.1134
17	000001	44	3831	0.2994	0.0830
18	000001	48	26040	0.3009	0.0799
19	000001	50	19180	0.4281	0.1520

	pseudo_reflectivity_mean	pseudo_reflectivity_std	range_mean	range_std
0	2.5678	6.4727	59.0406	11.1369
1	2.5537	4.0309	14.4423	9.1182
2	1.8950	1.4602	9.7795	6.5467
3	3.8336	2.2884	13.4681	7.9850
4	2.4723	1.4484	8.5185	5.2274
5	6.2930	2.6919	15.3992	6.6948
6	6.0659	2.4863	21.3816	2.1585
7	4.1216	2.2717	13.6191	6.9195
8	2.6839	0.9981	7.0933	2.6144
9	4.8617	2.7450	16.4867	10.3004
10	6.8026	2.9460	24.8873	9.1963
11	6.4119	2.5194	22.9273	9.1091
12	2.4216	2.8849	11.4776	8.7602
13	4.7278	3.7913	21.0138	12.5094
14	2.7169	7.0997	59.5173	11.1551
15	2.4465	3.7950	14.1880	9.4371
16	1.9004	1.4638	9.9190	6.7176
17	3.5215	2.2271	12.3583	

```
7.7030
18                    2.4490                    1.4421      8.4375
5.2630
19                    6.2019                    2.6865     15.1860
6.6215
```

Step 5: Keep only temporally persistent classes

In this step, we keep only those classes that survive across all selected frames.

This gives us a clean basis for short-window semantic consistency analysis.

```
num_selected_frames = len(frame_ids)

class_frame_counts = (
    class_summary_df.groupby("class_id")["frame_id"]
    .nunique()
    .reset_index(name="num_frames_present")
)

persistent_classes = class_frame_counts.loc[
    class_frame_counts["num_frames_present"] == num_selected_frames,
    "class_id"
].tolist()

persistent_summary_df = class_summary_df[
    class_summary_df["class_id"].isin(persistent_classes)
].copy().reset_index(drop=True)

print("Selected frames:", num_selected_frames)
print("Persistent classes:", persistent_classes)
print("Number of persistent classes:", len(persistent_classes))
print("Persistent summary shape:", persistent_summary_df.shape)

class_frame_counts.sort_values(["num_frames_present", "class_id"],
ascending=[False, True])
```

```
Selected frames: 10
Persistent classes: [0, 10, 40, 44, 48, 50, 51, 52, 60, 70, 71, 72,
80, 99]
Number of persistent classes: 14
Persistent summary shape: (140, 9)
```

```
    class_id  num_frames_present
0          0                  10
1         10                  10
2         40                  10
3         44                  10
4         48                  10
5         50                  10
```

```
6          51                    10
7          52                    10
8          60                    10
9          70                    10
10         71                    10
11         72                    10
12         80                    10
13         99                    10
```

Step 6: Measure class-wise temporal stability

In this step, we quantify how much each class mean changes across the selected motion window for both raw intensity and pseudo-reflectivity.

This gives us a first direct measure of short-horizon semantic consistency.

```
stability_rows = []

for class_id in persistent_classes:
    cls_df = persistent_summary_df[persistent_summary_df["class_id"]
== class_id].sort_values("frame_id")

    intensity_means = cls_df["intensity_mean"].values
    pseudo_means = cls_df["pseudo_reflectivity_mean"].values

    stability_rows.append({
        "class_id": int(class_id),
        "num_frames": len(cls_df),
        "intensity_mean_over_time": float(np.mean(intensity_means)),
        "intensity_std_over_time": float(np.std(intensity_means)),
        "intensity_cv_over_time": float(np.std(intensity_means) /
(np.mean(intensity_means) + 1e-12)),
        "pseudo_mean_over_time": float(np.mean(pseudo_means)),
        "pseudo_std_over_time": float(np.std(pseudo_means)),
        "pseudo_cv_over_time": float(np.std(pseudo_means) /
(np.mean(pseudo_means) + 1e-12)),
    })

stability_df =
pd.DataFrame(stability_rows).sort_values("class_id").reset_index(drop=
True)

stability_df
```

```
    class_id  num_frames  intensity_mean_over_time
intensity_std_over_time  \
0          0          10                    0.0468
0.0030
1         10          10                    0.1290
```

```
0.0163
2         40          10                    0.2118
0.0013
3         44          10                    0.3024
0.0021
4         48          10                    0.3002
0.0009
5         50          10                    0.4324
0.0067
6         51          10                    0.3187
0.0166
7         52          10                    0.3674
0.0209
8         60          10                    0.3963
0.0197
9         70          10                    0.3245
0.0048
10        71          10                    0.3125
0.0039
11        72          10                    0.2964
0.0134
12        80          10                    0.1798
0.0138
13        99          10                    0.3015
0.0255

    intensity_cv_over_time  pseudo_mean_over_time
pseudo_std_over_time  \
0                   0.0631                 2.6052
0.1384
1                   0.1261                 1.8978
0.4232
2                   0.0060                 1.9459
0.0343
3                   0.0070                 2.9334
0.4198
4                   0.0030                 2.5255
0.0974
5                   0.0155                 6.0199
0.1353
6                   0.0522                 5.8411
0.2264
7                   0.0569                 5.2178
0.6119
8                   0.0496                 3.4241
0.7067
9                   0.0147                 4.4751
0.1926
10                  0.0125                 5.9944
```

```
0.6117
11                    0.0451                    6.8468
0.2338
12                    0.0766                    1.8447
0.2957
13                    0.0846                    5.5547
0.9460

    pseudo_cv_over_time
0                0.0531
1                0.2230
2                0.0176
3                0.1431
4                0.0386
5                0.0225
6                0.0388
7                0.1173
8                0.2064
9                0.0430
10               0.1020
11               0.0341
12               0.1603
13               0.1703
```

Step 7: Test whether class ordering persists across frames

In this step, we compare the class-wise ordering of frames against the average class ordering over the whole window.

This checks whether the semantic structure remains consistent through motion, even if exact values drift.

```
from scipy.stats import spearmanr

# Pivot tables: rows = frame_id, columns = class_id
intensity_pivot = persistent_summary_df.pivot(
    index="frame_id", columns="class_id", values="intensity_mean"
).sort_index()

pseudo_pivot = persistent_summary_df.pivot(
    index="frame_id", columns="class_id",
values="pseudo_reflectivity_mean"
).sort_index()

# Reference profile = average class means across the full frame window
intensity_reference = intensity_pivot.mean(axis=0)
pseudo_reference = pseudo_pivot.mean(axis=0)

ordering_rows = []
```

```
for frame_id in intensity_pivot.index:
    intensity_corr, _ = spearmanr(intensity_pivot.loc[frame_id],
intensity_reference)
    pseudo_corr, _ = spearmanr(pseudo_pivot.loc[frame_id],
pseudo_reference)

    ordering_rows.append({
        "frame_id": frame_id,
        "intensity_rank_corr_vs_mean_profile": float(intensity_corr),
        "pseudo_rank_corr_vs_mean_profile": float(pseudo_corr),
    })

ordering_df = pd.DataFrame(ordering_rows)

print("Class-ordering consistency across frames")
display(ordering_df)

print("\nAverage rank correlation")
print("Intensity        :",
round(ordering_df["intensity_rank_corr_vs_mean_profile"].mean(), 4))
print("Pseudo-reflectivity:",
round(ordering_df["pseudo_rank_corr_vs_mean_profile"].mean(), 4))
```

```
Class-ordering consistency across frames

  frame_id  intensity_rank_corr_vs_mean_profile  \
0   000000                                0.8901
1   000001                                0.9165
2   000002                                0.9297
3   000003                                0.9648
4   000004                                0.9824
5   000005                                0.9868
6   000006                                0.9912
7   000007                                0.9648
8   000008                                0.9516
9   000009                                0.9560

   pseudo_rank_corr_vs_mean_profile
0                            0.9473
1                            0.9560
2                            0.9736
3                            0.9868
4                            0.9868
5                            0.9956
6                            0.9473
7                            0.9385
8                            0.9253
9                            0.9253
```

```
Average rank correlation
Intensity        : 0.9534
Pseudo-reflectivity: 0.9582
```

Interpretation of the ordering-consistency result

The class-wise ordering remains highly stable across the 10-frame motion window for both raw intensity and pseudo-reflectivity. This means the semantic structure observed earlier is not just a one-frame accident.

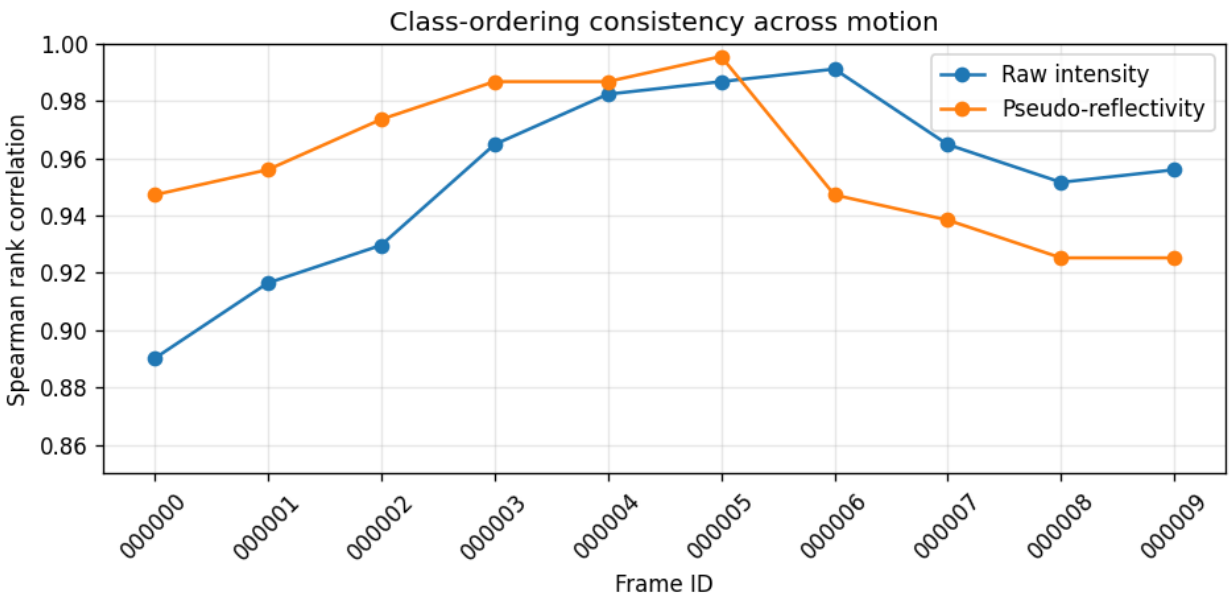
Pseudo-reflectivity achieves a slightly higher average rank correlation than raw intensity, which suggests that its class-level relationships persist at least as well, and here marginally better, through short-horizon motion. The gain is modest, so the correct conclusion is not dominance, but **short-window semantic consistency with mild advantage**.

Step 8: Visualize ordering consistency across motion

In this step, we plot the frame-wise rank-correlation values for raw intensity and pseudo-reflectivity.

This gives a compact visual check of how stable the semantic class ordering remains as the platform moves.

```
plt.figure(figsize=(8, 4))

plt.plot(
    ordering_df["frame_id"],
    ordering_df["intensity_rank_corr_vs_mean_profile"],
    marker="o",
    label="Raw intensity"
)

plt.plot(
    ordering_df["frame_id"],
    ordering_df["pseudo_rank_corr_vs_mean_profile"],
    marker="o",
    label="Pseudo-reflectivity"
)

plt.ylim(0.85, 1.00)
plt.xlabel("Frame ID")
plt.ylabel("Spearman rank correlation")
plt.title("Class-ordering consistency across motion")
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Step 9: Summarize the consistency result numerically

In this step, we compute a few compact summary numbers from the frame-wise rank correlations.

This gives us a neat final quantitative statement for Notebook 05 without bloating the analysis.

```
intensity_vals =
ordering_df["intensity_rank_corr_vs_mean_profile"].values
pseudo_vals = ordering_df["pseudo_rank_corr_vs_mean_profile"].values

summary_05 = {
    "num_frames": len(ordering_df),
    "intensity_mean_rank_corr": float(np.mean(intensity_vals)),
    "pseudo_mean_rank_corr": float(np.mean(pseudo_vals)),
    "intensity_min_rank_corr": float(np.min(intensity_vals)),
    "pseudo_min_rank_corr": float(np.min(pseudo_vals)),
    "intensity_max_rank_corr": float(np.max(intensity_vals)),
    "pseudo_max_rank_corr": float(np.max(pseudo_vals)),
    "pseudo_better_frames": int(np.sum(pseudo_vals > intensity_vals)),
    "intensity_better_frames": int(np.sum(intensity_vals >
pseudo_vals)),
    "tied_frames": int(np.sum(np.isclose(pseudo_vals,
intensity_vals))),
    "mean_advantage_pseudo_minus_intensity": float(np.mean(pseudo_vals
- intensity_vals)),
}

for k, v in summary_05.items():
    if isinstance(v, float):
        print(f"{k}: {v:.4f}")
```

```
    else:
        print(f"{k}: {v}")
```

```
num_frames: 10
intensity_mean_rank_corr: 0.9534
pseudo_mean_rank_corr: 0.9582
intensity_min_rank_corr: 0.8901
pseudo_min_rank_corr: 0.9253
intensity_max_rank_corr: 0.9912
pseudo_max_rank_corr: 0.9956
pseudo_better_frames: 6
intensity_better_frames: 4
tied_frames: 0
mean_advantage_pseudo_minus_intensity: 0.0048
```

Step 10: Generate a labeled 30-frame reflectivity-augmented semantic GIF

In this step, we create a short motion GIF where semantic labels are shown with explicit legend information, while pseudo-reflectivity modulates the brightness of each class.

This makes the semantic identity visible frame-by-frame instead of relying on color alone.

```
from pathlib import Path
import tempfile
import imageio.v2 as imageio
from IPython.display import Image, display
from matplotlib.patches import Patch

# 1) 30-frame window

START_FRAME_GIF = 0
NUM_FRAMES_GIF = 30
gif_frame_ids = [f"{i:06d}" for i in range(START_FRAME_GIF,
START_FRAME_GIF + NUM_FRAMES_GIF)]

# 2) SemanticKITTI color map

SEMANTIC_COLOR_MAP = {
    0:   (0.00, 0.00, 0.00),       # unlabeled
    10:  (0.96, 0.59, 0.39),       # car
    40:  (1.00, 0.00, 1.00),       # road
    44:  (1.00, 0.59, 1.00),       # parking
    48:  (0.29, 0.00, 0.29),       # sidewalk
    49:  (0.29, 0.00, 0.69),       # other-ground
    50:  (0.00, 0.00, 1.00),       # building
    51:  (0.67, 1.00, 1.00),       # fence
    52:  (0.00, 0.67, 1.00),       # other-structure
    60:  (1.00, 1.00, 0.00),       # lane-marking
```

```
    70:  (0.00, 0.69, 0.00),       # vegetation
    71:  (0.00, 0.47, 0.00),       # trunk
    72:  (0.00, 0.86, 0.00),       # terrain
    80:  (1.00, 0.94, 0.59),       # pole
    81:  (1.00, 0.71, 0.59),       # traffic-sign
    99:  (1.00, 1.00, 0.31),       # other-object
    252: (0.53, 0.81, 0.98),       # moving-car
    253: (1.00, 0.71, 0.59),       # moving-bicyclist
    254: (1.00, 0.47, 0.39),       # moving-person
    255: (1.00, 0.00, 0.00),       # moving-object
}

CLASS_NAME_MAP = {
    0: "unlabeled",
    10: "car",
    40: "road",
    44: "parking",
    48: "sidewalk",
    49: "other-ground",
    50: "building",
    51: "fence",
    52: "other-structure",
    60: "lane-marking",
    70: "vegetation",
    71: "trunk",
    72: "terrain",
    80: "pole",
    81: "traffic-sign",
    99: "other-object",
    252: "moving-car",
    253: "moving-bicyclist",
    254: "moving-person",
    255: "moving-object",
}

def semantic_rgb(labels):
    rgb = np.zeros((len(labels), 3), dtype=np.float32)
    for cid in np.unique(labels):
        rgb[labels == cid] = SEMANTIC_COLOR_MAP.get(int(cid), (0.7,
0.7, 0.7))
    return rgb

def reflectivity_augmented_rgb(labels, pseudo_reflectivity):
    base_rgb = semantic_rgb(labels)

    lo = np.percentile(pseudo_reflectivity, 5)
    hi = np.percentile(pseudo_reflectivity, 95)
    pr_norm = (pseudo_reflectivity - lo) / (hi - lo + 1e-12)
    pr_norm = np.clip(pr_norm, 0.0, 1.0)
```

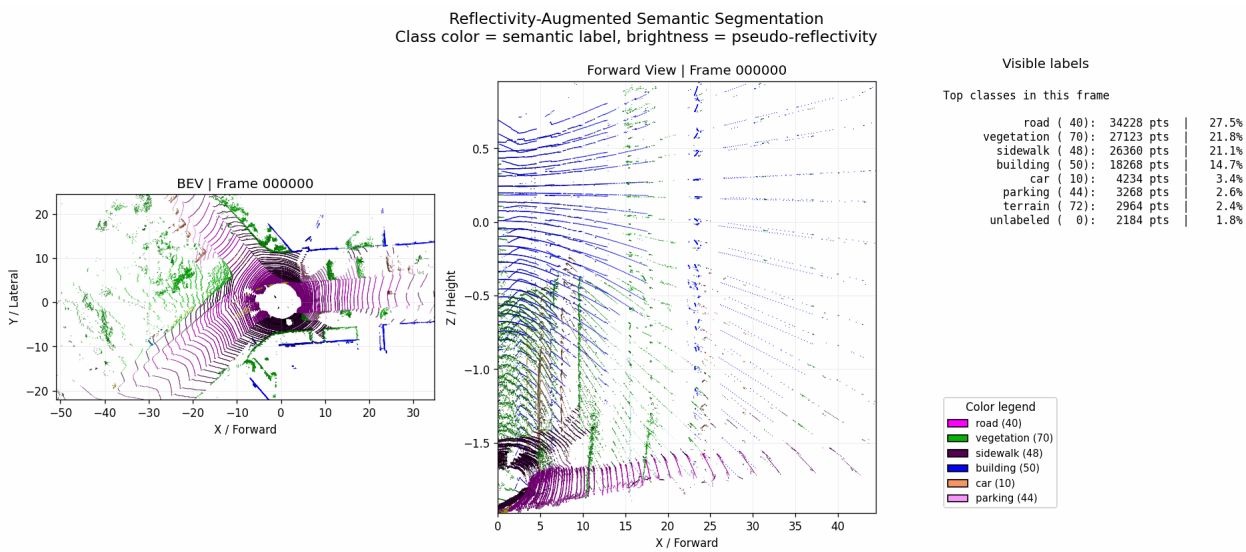
```
    brightness = 0.35 + 0.65 * pr_norm
    return np.clip(base_rgb * brightness[:, None], 0.0, 1.0)

# 3) Compute fixed view limits from all 30 frames

all_xyz_list = []

for fid in gif_frame_ids:
    xyz, intensity = load_frame_points(fid)
    all_xyz_list.append(xyz)

all_xyz = np.vstack(all_xyz_list)

# BEV limits
x_min, x_max = np.percentile(all_xyz[:, 0], [1, 99])
y_min, y_max = np.percentile(all_xyz[:, 1], [1, 99])

# Forward-view limits
all_x = all_xyz[:, 0]
all_z = all_xyz[:, 2]
forward_mask = all_x > 0.0

fov_x_min = 0.0
fov_x_max = float(np.percentile(all_x[forward_mask], 99))
fov_z_min = float(np.percentile(all_z[forward_mask], 1))
fov_z_max = float(np.percentile(all_z[forward_mask], 99))

# 4) Output path

artifacts_dir = Path("artifacts")
artifacts_dir.mkdir(exist_ok=True)

gif_path = artifacts_dir /
"notebook05_reflectivity_augmented_semantic_motion_30f_labeled.gif"
fps = 3  # 30 frames -> 10 seconds

# 5) Render frames

rendered_frame_paths = []

with tempfile.TemporaryDirectory() as tmpdir:
    temp_dir = Path(tmpdir)

    for idx, fid in enumerate(gif_frame_ids):
        xyz, intensity = load_frame_points(fid)
        labels = load_frame_labels(fid)
        ranges, pr = compute_range_and_pseudo_reflectivity(xyz,
intensity)

        x = xyz[:, 0]
        y = xyz[:, 1]
```

```
        z = xyz[:, 2]
        fwd = x > 0.0

        aug_rgb = reflectivity_augmented_rgb(labels, pr)

        uniq, cnts = np.unique(labels, return_counts=True)
        order = np.argsort(cnts)[::-1]

        top_class_ids = [int(uniq[k]) for k in order[:6]]
        top_legend_handles = [
            Patch(
                facecolor=SEMANTIC_COLOR_MAP.get(cid, (0.7, 0.7,
0.7)),
                edgecolor="black",
                label=f"{CLASS_NAME_MAP.get(cid, f'class-{cid}')}
({cid})"
            )
            for cid in top_class_ids
        ]

        fig = plt.figure(figsize=(16, 7))
        gs = fig.add_gridspec(1, 3, width_ratios=[1.15, 1.15, 0.65])

        ax_bev = fig.add_subplot(gs[0, 0])
        ax_fwd = fig.add_subplot(gs[0, 1])
        ax_info = fig.add_subplot(gs[0, 2])

        # BEV
        ax_bev.scatter(
            x, y,
            s=0.35,
            c=aug_rgb,
            linewidths=0
        )
        ax_bev.set_title(f"BEV | Frame {fid}")
        ax_bev.set_xlim(x_min, x_max)
        ax_bev.set_ylim(y_min, y_max)
        ax_bev.set_aspect("equal")
        ax_bev.set_xlabel("X / Forward")
        ax_bev.set_ylabel("Y / Lateral")
        ax_bev.grid(True, alpha=0.2)

        # Forward view
        ax_fwd.scatter(
            x[fwd], z[fwd],
            s=0.5,
            c=aug_rgb[fwd],
            linewidths=0
        )
        ax_fwd.set_title(f"Forward View | Frame {fid}")
```

```
        ax_fwd.set_xlim(fov_x_min, fov_x_max)
        ax_fwd.set_ylim(fov_z_min, fov_z_max)
        ax_fwd.set_xlabel("X / Forward")
        ax_fwd.set_ylabel("Z / Height")
        ax_fwd.grid(True, alpha=0.2)

        # Info / labels panel
        ax_info.axis("off")
        ax_info.set_title("Visible labels", pad=12)

        info_lines = []
        total_points = len(labels)
        for k in order[:8]:
            cid = int(uniq[k])
            cname = CLASS_NAME_MAP.get(cid, f"class-{cid}")
            frac = 100.0 * cnts[k] / total_points
            info_lines.append(f"{cname:>16} ({cid:>3}): {cnts[k]:>6}
pts  |  {frac:5.1f}%")

        ax_info.text(
            0.02, 0.98,
            "Top classes in this frame\n\n" + "\n".join(info_lines),
            va="top",
            ha="left",
            fontsize=10,
            family="monospace"
        )

        ax_info.legend(
            handles=top_legend_handles,
            loc="lower left",
            frameon=True,
            fontsize=9,
            title="Color legend"
        )

        fig.suptitle(
            "Reflectivity-Augmented Semantic Segmentation\n"
            "Class color = semantic label, brightness = pseudo-
reflectivity",
            fontsize=14
        )

        plt.tight_layout()

        frame_png = temp_dir / f"frame_{idx:03d}.png"
        plt.savefig(frame_png, dpi=120, bbox_inches="tight")
        plt.close(fig)

        rendered_frame_paths.append(frame_png)
```

```
    images = [imageio.imread(p) for p in rendered_frame_paths]
    imageio.mimsave(gif_path, images, fps=fps)

print("GIF created at:", gif_path.resolve())
print("Total frames:", len(rendered_frame_paths))
print("Approx duration (s):", len(rendered_frame_paths) / fps)

display(Image(filename=str(gif_path)))
```

```
GIF created at:
/home/twilightpriest/GitHub/reflect-aug-seg/notebooks/artifacts/notebo
ok05_reflectivity_augmented_semantic_motion_30f_labeled.gif
Total frames: 30
Approx duration (s): 10.0
```



Interpretation of the labeled semantic motion GIF

This GIF provides a qualitative check that complements the temporal consistency metrics. Across the 30-frame motion window, the dominant semantic classes remain visually coherent in both bird's-eye and forward-view projections, while the explicit label panel shows how the scene composition evolves from frame to frame.

The important point is that the visualization is not a plain semantic segmentation display. Each point keeps its semantic class color, but its brightness is modulated by pseudo-reflectivity. This makes it possible to see whether the range-aware signal introduces structured variation within and across semantic classes as the platform moves.

The result supports the same restrained conclusion suggested by the quantitative analysis: the semantic patterns are not a one-frame accident, and the reflectivity-aware proxy remains visually meaningful across short-horizon motion. At the same time, the effect is not uniform across all classes or all frames, which is consistent with the earlier observation that the proxy offers a mild and scene-dependent advantage rather than universal dominance.

Final conclusion

This notebook tested whether the class-wise semantic patterns observed in the single-frame analysis persist across short-horizon motion. Using 10 consecutive frames for quantitative evaluation and a 30-frame labeled GIF for qualitative inspection, the results show that the semantic structure remains strongly consistent over motion rather than appearing as a one-frame lucky strike.

At the quantitative level, both raw intensity and pseudo-reflectivity preserved class ordering with very high rank correlation across the selected frame window. Raw intensity achieved an average class-ordering consistency of `0.9534`, while pseudo-reflectivity achieved `0.9582`. Pseudo-reflectivity also produced the stronger worst-case frame (`0.9253` vs `0.8901`) and performed better in 6 out of 10 frames. This indicates a mild overall advantage for the range-aware proxy, but not a dramatic or universal one.

At the qualitative level, the labeled 30-frame reflectivity-augmented semantic GIF showed that dominant scene classes remained visually coherent across motion in both bird's-eye and forward-view projections. Because semantic class identity was preserved through color and pseudo-reflectivity was introduced through brightness modulation, the visualization made it clear that the proxy contributes structured within-class and across-class variation rather than random visual noise.

The correct preliminary conclusion is therefore:

the semantic class structure associated with the range-aware pseudo-reflectivity proxy is not a one-frame coincidence. It remains strongly consistent across short motion windows, shows a mild overall advantage over raw intensity in preserving class-wise ordering, and appears visually meaningful over motion, while still exhibiting scene-dependent behavior that should be examined more broadly in the final stage.

This is an appropriate stopping point for the preliminary phase. It validates short-window semantic consistency without exhausting the larger final-project agenda, which should still include broader temporal coverage, stronger class-wise metrics, robustness checks, alternative proxy formulations, and failure-case analysis.